API folder guide

lib/api/ — which file does what, in the Amber frontend

Core files (everything else depends on these)

File	What it does
client.ts	The engine for all API calls. **API_BASE** is the server URL (from NEXT_PUBLIC_API_URL, default is the Vercel URL). **call() / post()** send the JSON request and check the response; if success is false they throw an **ApiError** so the page can show the message. **currentUserId()** returns the signed-in user's id, because the API needs user_id.
types.ts	The shape of every response (TypeScript types), e.g. Building, Vendor, ComplaintRow, ComplaintDetail, AdminDashboard, PmDashboard. Gives autocomplete and catches wrong field names.
mappers.ts	Converts API data into the format the UI expects. roleKeyFromApi() maps project_manager to the PM panel and client to the Creator panel. statusFromApi() maps PENDING_APPROVAL to pending_verification. priorityFromType() maps Emergency to Critical. toComplaint() and toComplaintFromDetail() build the Complaint object used by pages. fmtDate() and timeAgo() format dates.

Endpoint files (one area, one file)

File	APIs
auth.ts	login
buildings.ts	getBuildings, createBuildingWithUser
vendors.ts	getVendors, getVendorWorkers
users.ts	createUser
complaint-types.ts	getComplaintTypes, getComplaintCategories, getBuildingComplaintTypes
complaints.ts	createComplaint, checkMedia, getComplaints, getComplaintDetail, assignWorker
worker.ts	workerCheckin, workerStart, workerSubmit
dashboard.ts	getAdminDashboard, getPmDashboard

To add a new API: write one function in the file for its area, then add it to index.ts.

The other two files

- **index.ts** — joins every endpoint file into one **api** object. Pages write `import { api } from "@/lib/api"` and call `api.getComplaints()`. It also re-exports types.ts and mappers.ts, so one import is enough.
- **README.md** — a short table of the same information, for quick reference.

Files outside the folder that support it

File	What it does
lib/session.ts	Saves the login token and user in the browser (localStorage).
lib/use-api.ts	Hook that pages use to call an API; handles loading, error and data.
lib/vendor-workers.ts	Normalises worker row field names.

Journey of one call

```
creator/new page  ->  api.createComplaint(...)
  ->  complaints.ts  ->  post() in client.ts  ->  server
  ->  response matches types.ts
  ->  mappers.ts converts it for the UI if needed
```